TIPS: SQL Template Queries

The SQL GUI provides a set of template queries accessible from the Templates dropdown menu. Each template is a ready-made SQL pattern with placeholder fields in [brackets]. When you click inside a placeholder and select a table or field from the dropdown menus, the placeholder is replaced — and if the same placeholder appears multiple times (e.g., [field1] in a COUNT query), all occurrences are replaced at once.

This document describes each template, shows its syntax, and explains when to use it.

Quick Reference

Template	Purpose
`SQL standard`	Basic SELECT with WHERE filter and ORDER BY
`SQL count`	Frequency counts with GROUP BY
`SQL duplicates`	Find records that share the same key value
`SQL join`	Inner join — combine two tables on a shared field
`SQL left join`	Join keeping all rows from Table1, even unmatched
`SQL unmatched`	Find records in Table1 with no match in Table2
`SQL union`	Stack results from two queries into one list
`SQL update`	Modify existing data in the database
`SQL subquery`	Filter rows using values from another table
`SQL group concat`	Aggregate multiple values into one cell
`SQL case`	Conditional if-then-else categorization

Template Details

SQL standard — Standard SELECT query

The most basic and commonly used SQL query. Retrieves specific columns from a table, filters rows with a WHERE condition, and sorts the output. This is the starting point for most data exploration tasks.

Syntax:

```
SELECT [Field1], [Field2], [Field3]
  FROM [table]
  WHERE [field2] = "string value"
  ORDER BY [Field1], [Field3] DESC;
```

When to use:

- Browse the contents of any table to understand its structure
- Filter data by a specific condition (e.g., all actors of type 'individual')

- Sort results alphabetically or numerically for review

SQL count — Frequency COUNT query

Counts how many times each unique value appears in a column. Essential for understanding the distribution of values in your data — which categories are most common, which are rare.

Syntax:

```
SELECT [field1], COUNT([field1]) AS FREQUENCY
  FROM [table]
  GROUP BY [field1]
  ORDER BY COUNT([field1]) DESC;
```

When to use:

- Count how many events of each type are in the database
- Find the most frequent actor names or locations
- Identify rare or potentially misspelled values (frequency = 1)

SQL duplicates — Find DUPLICATE records

Identifies records that share the same value in a key field. Useful for data quality checks — finding entries that may have been coded twice or values that should be unique but are not.

Syntax:

```
SELECT [1].[field1], [1].[field2], [1].[field3]
  FROM [table] AS [1]
  INNER JOIN (
    SELECT [table].[Field1]
    GROUP BY [table].[Field1]
    HAVING (COUNT([table].[Field1]) > 1)
  ) AS [2] ON [1].[Field1]=[2].[Field1]
  ORDER BY [1].[Field1] DESC;
```

When to use:

- Find actors or events that may have been entered twice
- Identify simplex values with duplicate text (potential data-entry errors)
- Quality-check identifier uniqueness across complex instances

SQL join — INNER JOIN — combine two tables

Combines rows from two tables where a shared field matches. Only rows with a match in both tables appear in the result. This is the foundation of relational queries — linking data across PC-ACE tables.

Syntax:

```
SELECT [Table1_Field1], [Table1_Field2]
  FROM [Table1]
```

```
JOIN [Table2]
  ON [Table1].[Table1_Field1]=[Table2].[Table2_Field1]
```

When to use:

- Link data_Complex to setup_Complex to see complex type names alongside IDs
- Join data_Simplex to data_SimplexText to see actual text values
- Connect any two tables that share a foreign key relationship

SQL left join — LEFT JOIN — keep unmatched rows

Like an inner join, but keeps ALL rows from the left (first) table even when there is no matching row in the right (second) table. Unmatched columns from the right table show as NULL.

Syntax:

```
SELECT [Table1].[field1], [Table2].[field2]
  FROM [Table1]
  LEFT JOIN [Table2]
    ON [Table1].[field1]=[Table2].[field1];
```

When to use:

- See all complex instances, including those with no linked simplexes
- Check which documents have no linked events or actors
- Preserve all source records while optionally enriching with data from another table

SQL unmatched — Find UNMATCHED records

Finds records in Table1 that have no corresponding match in Table2. This is a LEFT JOIN with a NULL check — it returns only the orphaned rows. Essential for finding missing links in the data.

Syntax:

```
SELECT [Table1].[field1], [Table1].[field2]
  FROM [Table1]
  LEFT JOIN [Table2]
    ON [Table1].[field1]=[Table2].[field1]
  WHERE [Table2].[field1] IS NULL;
```

When to use:

- Find complex instances with no linked documents (missing sources)
- Identify simplexes that are defined in setup but never used in data
- Locate orphaned records after data cleanup or migration

SQL union — UNION — combine results from two queries

Stacks the results of two SELECT queries vertically into a single result set. Both queries must return the same number of columns. UNION removes duplicates; use UNION ALL to keep them.

Syntax:

```
SELECT [Table1_Field] AS Field1
  FROM [Table1]

UNION

SELECT [Table2_Field] AS Field1
  FROM [Table2]
  ORDER BY Field1;
```

When to use:

- Combine actor names from two different complex types into one list
- Merge text values from different simplex types for comparison
- Create a unified list from separate tables for export or charting

SQL update — UPDATE — modify existing data

Changes existing values in a table. The SET clause specifies the new value; the WHERE clause controls which rows are affected. Without a WHERE clause, ALL rows would be updated — always include one.

Syntax:

```
UPDATE [table]
  SET [field] = "new value"
  WHERE [field2] = "condition";
```

When to use:

- Backfill missing newspaper names in Italian databases where the name is implied
- Correct misspelled simplex values across all affected rows
- Standardize inconsistent data entries (e.g., 'NY' → 'New York')

Important: UPDATE only modifies the SQLite database. The source xlsx/pkl files are NOT changed. If you rebuild the SQLite from source files, the update will be lost.

SQL subquery — Subquery with IN — filter by another table

Uses a nested SELECT inside a WHERE clause to filter rows based on values found in another table. This is a powerful way to ask questions like 'show me all X that also appear in Y.'

Syntax:

```
SELECT [field1], [field2]
  FROM [Table1]
  WHERE [field1] IN (
    SELECT [field1] FROM [Table2]
  );
```

When to use:

- Find all complex instances whose IDs appear in a cross-reference table
- Filter simplex values to only those linked to a specific complex type
- Select documents that are referenced by at least one event or actor

SQL group concat — GROUP_CONCAT — aggregate values into one cell

Collects multiple values from different rows and concatenates them into a single semicolon-separated string, grouped by a key field. Useful for creating summary views.

Syntax:

```
SELECT [field1],
  GROUP_CONCAT([field2], '; ') AS Combined
  FROM [table]
  GROUP BY [field1]
  ORDER BY [field1];
```

When to use:

- List all simplex values associated with each complex instance in one row
- Summarize all newspaper names linked to each document
- Create compact reports where multiple attributes are shown per record

SQL case — CASE — conditional categorization

Applies if-then-else logic inside a query to recode or categorize values. Creates a new virtual column based on conditions. Useful for grouping values into broader categories for analysis.

Syntax:

```
SELECT [field1],
  CASE
    WHEN [field2] = 'value1' THEN 'Category A'
    WHEN [field2] = 'value2' THEN 'Category B'
    ELSE 'Other'
  END AS Category
  FROM [table];
```

When to use:

- Recode actor types into broader categories (e.g., 'mob', 'posse', 'crowd' → 'Collective')
- Create time period bins from date values (e.g., 1900-1910, 1911-1920)
- Flag specific values for review (e.g., mark empty or suspicious entries)

Using Placeholders

All template queries use [brackets] around placeholder names. When you position your cursor inside a placeholder and select a table or field from the dropdown menus:

- The entire placeholder (including brackets) is replaced with the selected name
- All occurrences of the same placeholder are replaced at once (case-insensitive)
- Different placeholders (e.g., [field1] vs [field2]) are independent — each can be replaced with a different field
- If your cursor is NOT inside a placeholder, the name is inserted at the cursor position

Example: In the COUNT template, [field1] appears 4 times. Click inside any [field1], select a field (e.g., Name), and all four are replaced with Name simultaneously.

WHERE Filter for Cross-Complex Queries

The cross-complex query generator includes a WHERE filter row that lets you restrict results by simplex values. The filter has three parts:

- Simplex — select which simplex attribute to filter on (e.g., Type of actor)
- Operator — choose LIKE (pattern match), = (exact match), != (exclude), or NOT LIKE
- Value — enter the filter value; for LIKE, use % as wildcard (e.g., %woman% matches any value containing 'woman')

The WHERE filter is optional. When left empty, the generated query returns all instances without filtering.